GRAPHS

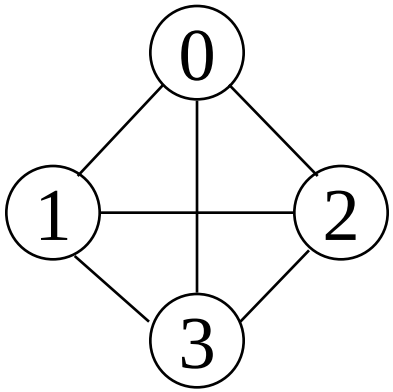
Definition

- A graph, *G=(V, E)*, consists of two sets:
  - a finite set of ***vertices(V)***, and
  - a finite, possibly empty set of edges***(E)***
  - *V(G)* and *E(G)* represent the sets of vertices and edges of *G*, respectively
- Undirected graph
  - The pairs of vertices representing any edges is ***unordered***
  - e.g., (v_0, v_1) and (v_1, v_0) represent the same edge

    (v0, v1) = (v1,v0)
- Directed graph
  - Each edge as a directed pair of vertices
  - e.g. $<v_0, v_1>$ represents an edge, v_0 is the tail and v_1 is the head

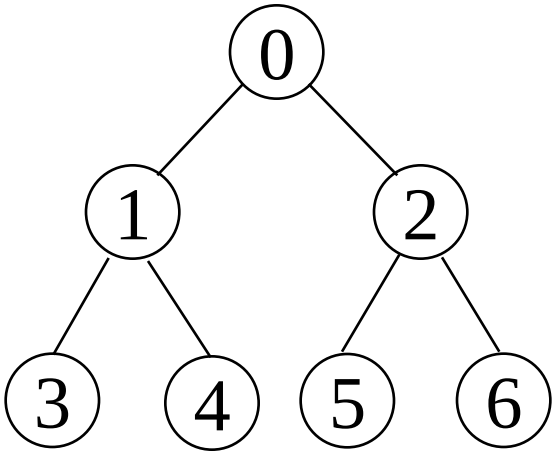
    <v0, v1> != <v1,v0>

Examples for Graph



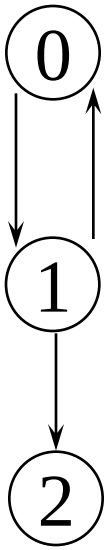
G_1

complete graph



G_2

incomplete graph



G_3

$V(G_1)=\{0,1,2,3\}$

$V(G_2)=\{0,1,2,3,4,5,6\}$

$V(G_3)=\{0,1,2\}$

$E(G_1)=\{(0,1),(0,2),(0,3),(1,2),(1,3),(2,3)\}$

$E(G_2)=\{(0,1),(0,2),(1,3),(1,4),(2,5),(2,6)\}$

$E(G_3)=\{<0,1>,<1,0>,<1,2>\}$

complete undirected graph: $n(n-1)/2$ edges

complete directed graph: $n(n-1)$ edges

Complete Graph

- A complete graph is a graph that has the maximum number of edges
  - for undirected graph with n vertices, the maximum number of edges is $n(n-1)/2$
  - for directed graph with n vertices, the maximum number of edges is $n(n-1)$
  - example: G1 is a complete graph

Undirected Graph

Notation. G = (V, E)

- V = nodes.
- E = edges between pairs of nodes.
- Captures pairwise relationship between objects.
- Graph size parameters: n = | V |, m = | E |.

V = { 1, 2, 3, 4, 5, 6, 7, 8 }
E = { 1-2, 1-3, 2-3, 2-4, 2-5, 3-5, 3-7, 3-8, 4-5, 5-6, 7-8 }
m = 11, n = 8

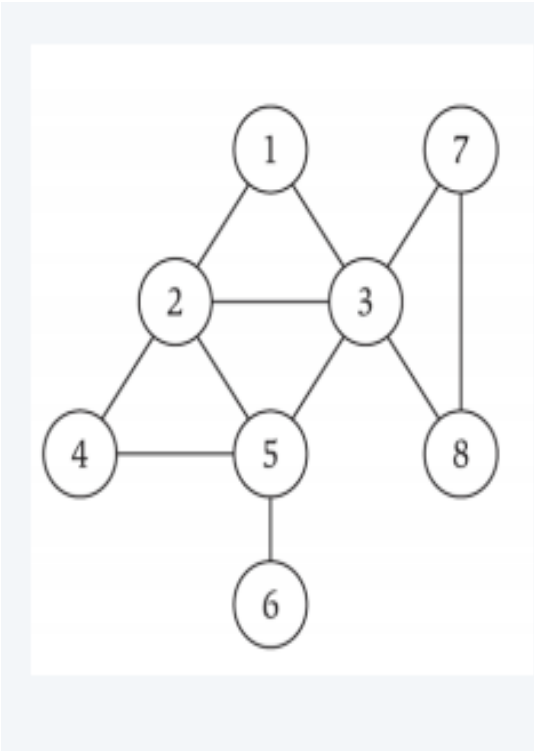
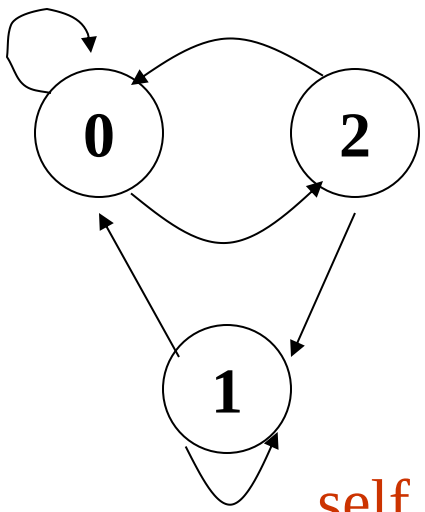
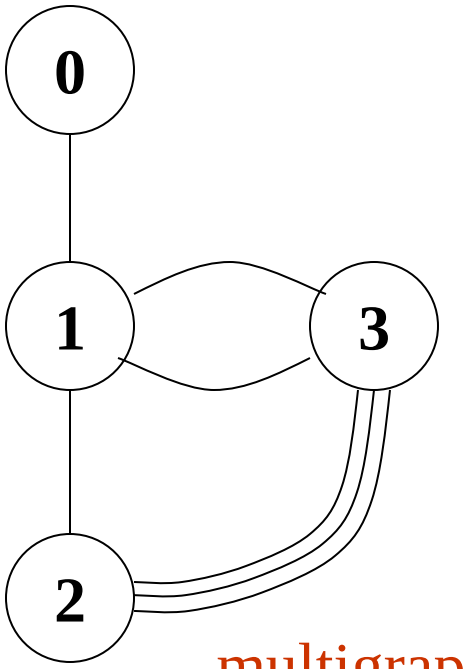


***Figure 6.3:** Example of a graph with feedback loops and a multigraph



self edge

(a)

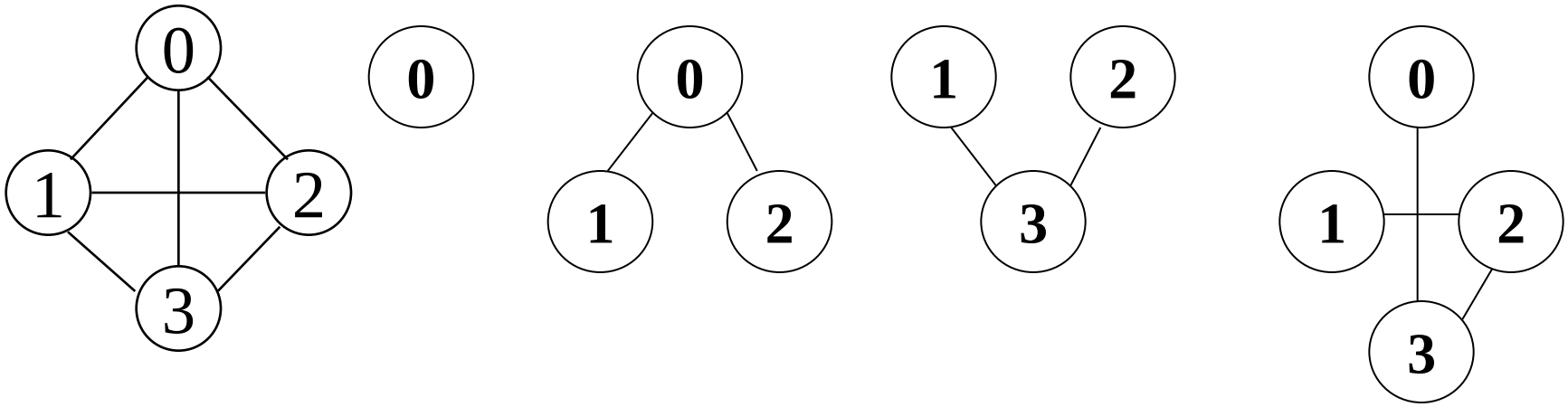


(b)

multigraph:
multiple occurrences
of the same edge

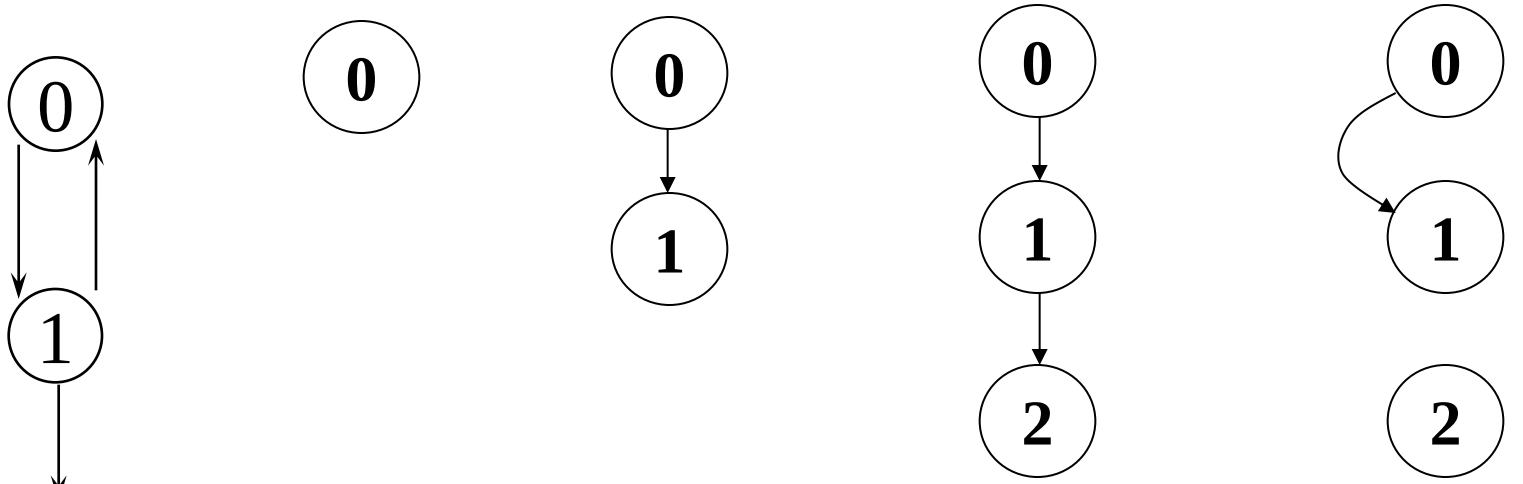
Figure 6.3

Figure 6.4: subgraphs of G_1 and G_3



G_1 (i) (ii) (iii) (iv)

(a) Some of the subgraph of G_1



(i) (ii) (iii) (iv)

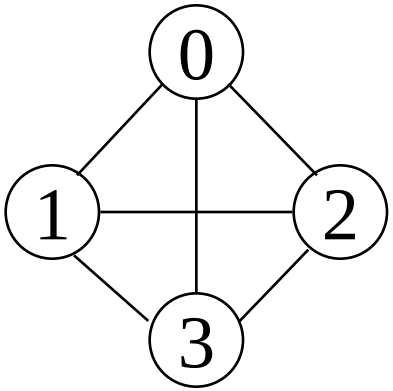
(b) Some of the subgraph of G_3

G_3

Path

- A simple path is a path in which all vertices, except possibly the first and the last, are distinct
- A cycle is a simple path in which the first and the last vertices are the same
- In an undirected graph G, two vertices, v0 and v1, are connected if there is a path in G from v0 to v1

connected



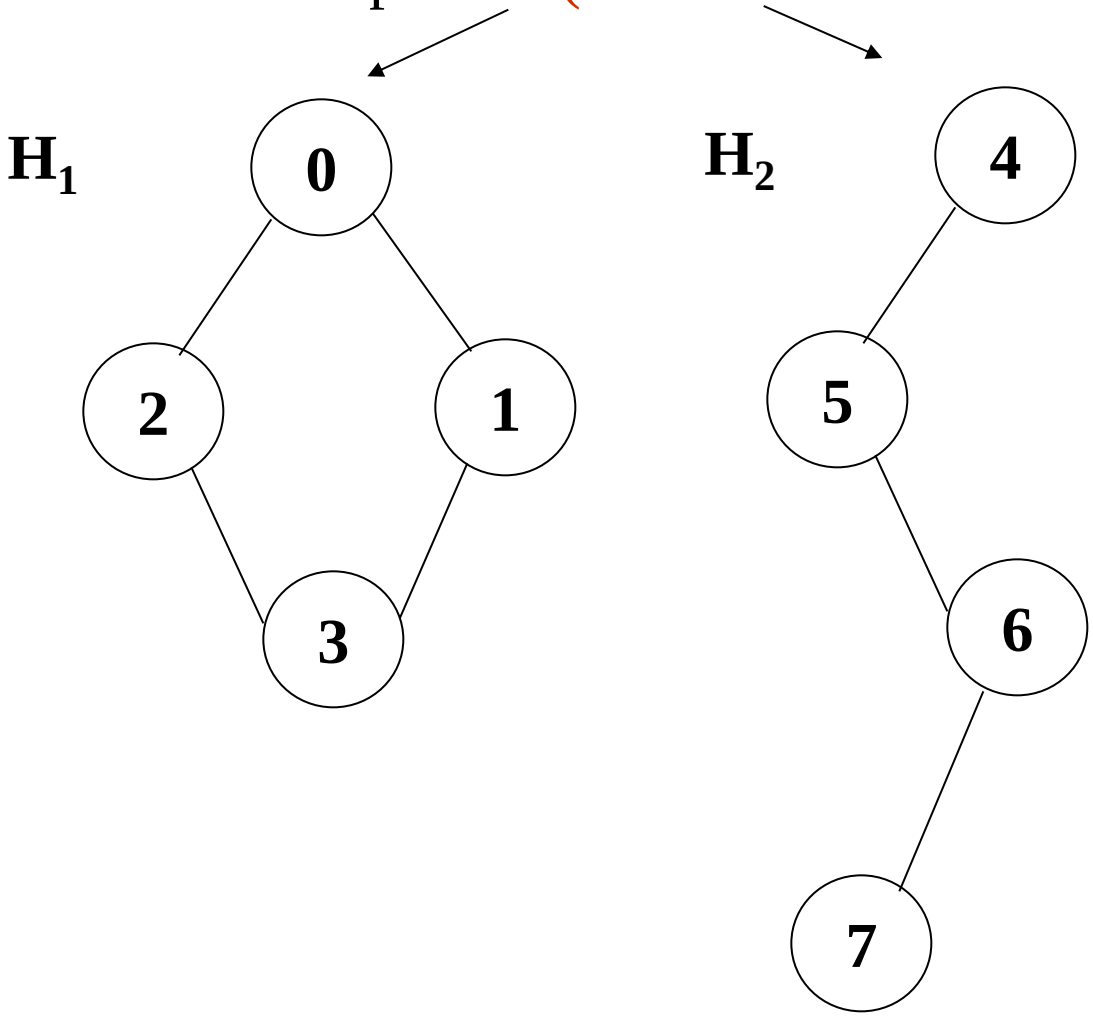
G_1

Connected Component

- A connected component of an undirected graph is a maximal connected sub graph.
- A tree is a graph that is connected and acyclic.
- A directed graph is strongly connected if there is a directed path from vi to vj and also from vj to vi.
- A strongly connected component is a maximal subgraph that is strongly connected.

***Figure 6.5:** A graph with two connected components

connected component (maximal connected subgraph)

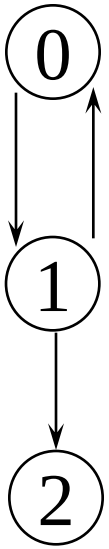


G_4 (not connected)

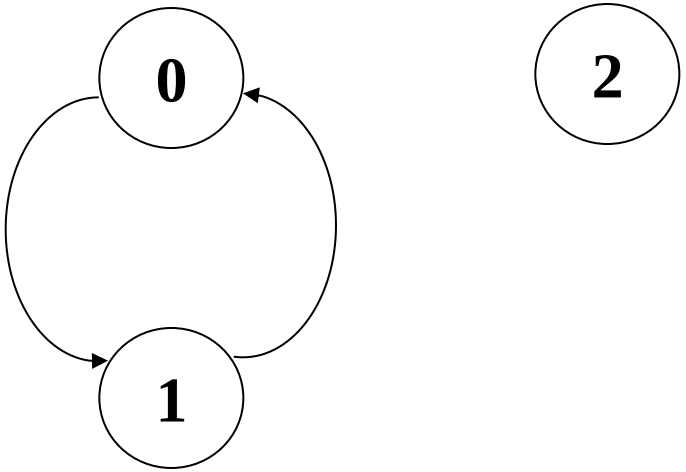
***Figure 6.6:** Strongly connected components of G_3

not strongly connected

strongly connected component
(maximal strongly connected subgraph)



G_3



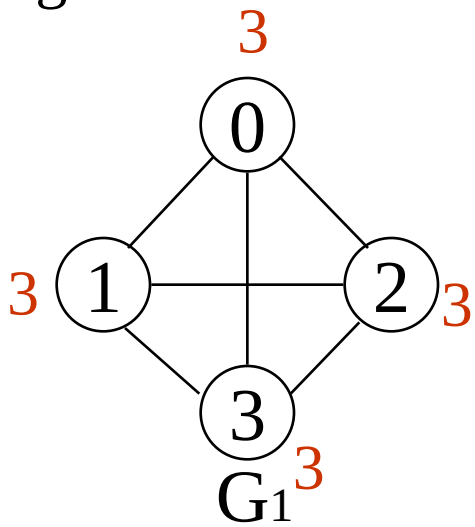
Degree

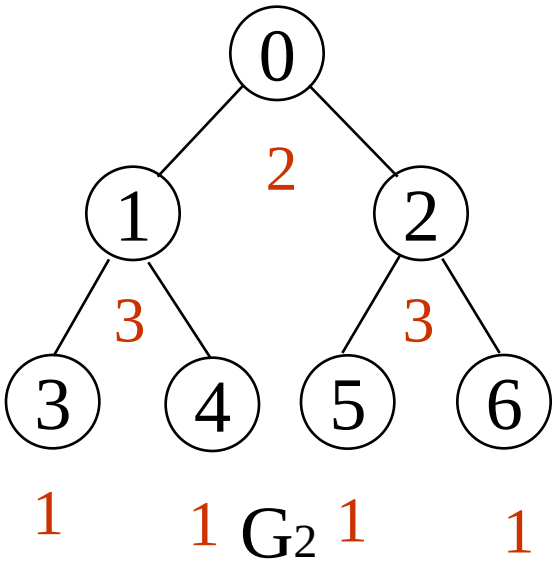
- The degree of a vertex is the number of edges incident to that vertex
- For directed graph,
  - the in-degree of a vertex *v* is the number of edges that have *v* as the head
  - the out-degree of a vertex *v* is the number of edges that have *v* as the tail
  - if *di* is the degree of a vertex *i* in a graph *G* with *n* vertices and *e* edges, the number of edges is

$$e = (\sum_{0}^{n-1} d_i) / 2$$

undirected graph

degree

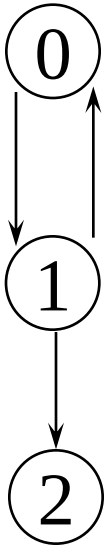




directed graph

in-degree

out-degree



in:1, out: 1

in: 1, out: 2

in: 1, out: 0

ADT for Graph

structure Graph is

objects: a nonempty set of vertices and a set of undirected edges, where each edge is a pair of vertices

functions: for all *graph* $\in$ *Graph*, *v*, v_1 and v_2 $\in$ *Vertices*

Graph Create()::=return an empty graph

Graph InsertVertex(*graph*, *v*)::= return a graph with *v* inserted. *v* has no incident edge.

Graph InsertEdge(*graph*, *v1*,*v2*)::= return a graph with new edge between *v1* and *v2*

Graph DeleteVertex(*graph*, *v*)::= return a graph in which *v* and all edges incident to it are removed

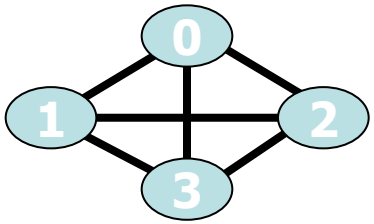
Graph DeleteEdge(*graph*, *v1*, *v2*)::=return a graph in which the edge (*v1*, *v2*) is removed

Boolean IsEmpty(*graph*)::= if (*graph==empty graph*) return TRUE
else return FALSE

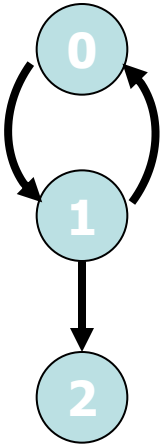
List Adjacent(*graph*,*v*)::= return a list of all vertices that are adjacent to *v*

Graphs Representation

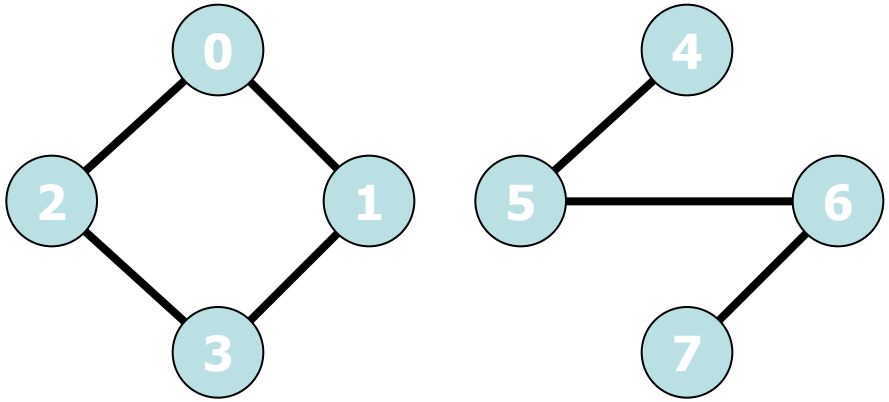
- Adjacency matrices
- Adjacency lists
- Adjacency multilists



G1



G3

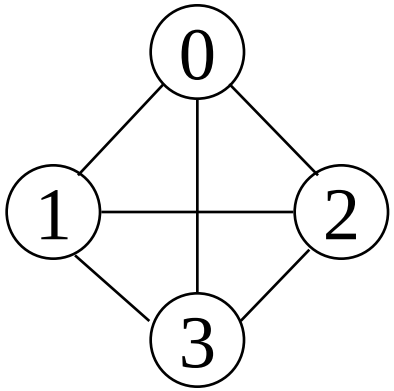


G4

Adjacency Matrix

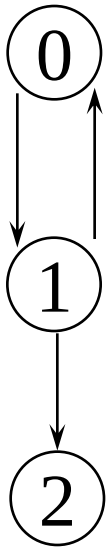
- Let G=(V,E) be a graph with n vertices.
- The adjacency matrix of G is a two-dimensional n by n array, say adj_mat
- If the edge (vi, vj) is in E(G), adj_mat[i][j]=1
- If there is no such edge in E(G), adj_mat[i][j]=0
- The adjacency matrix for an undirected graph is symmetric; the adjacency matrix for a digraph need not be symmetric

Examples for Adjacency Matrix



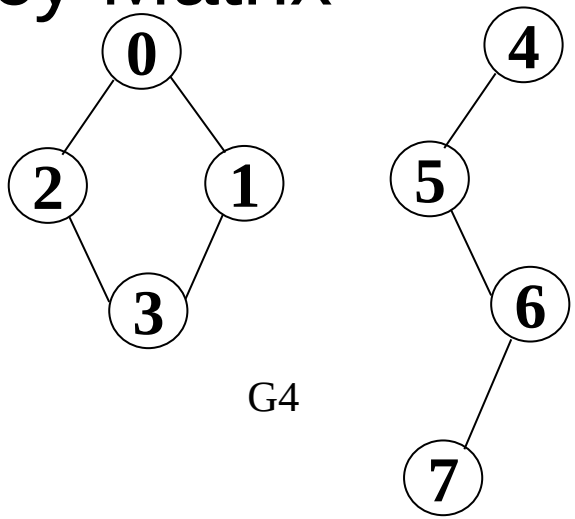
$$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

G_1



$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

G_2



G4

symmetric

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

G_4

Merits of Adjacency Matrix

- From the adjacency matrix, to determine the connection of vertices is easy
- The degree of a vertex is $\sum_{j=0}^{n-1} adj_mat[i][j]$
- For a digraph, the row sum is the out_degree, while the column sum is the in_degree

$$ind(vi) = \sum_{j=0}^{n-1} A[j,i] \qquad outd(vi) = \sum_{j=0}^{n-1} A[i,j]$$

Adjacency lists

- linked list

```
struct node {
        int vertex;
        node  *link;
};
```

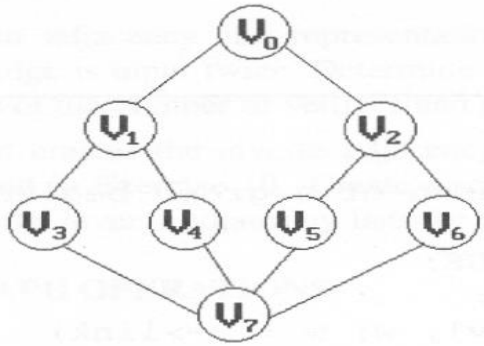
```
node *graphNodes[MAX_VERTICES];
int n = 0; /* number of nodes */
```

Some Graph Operations

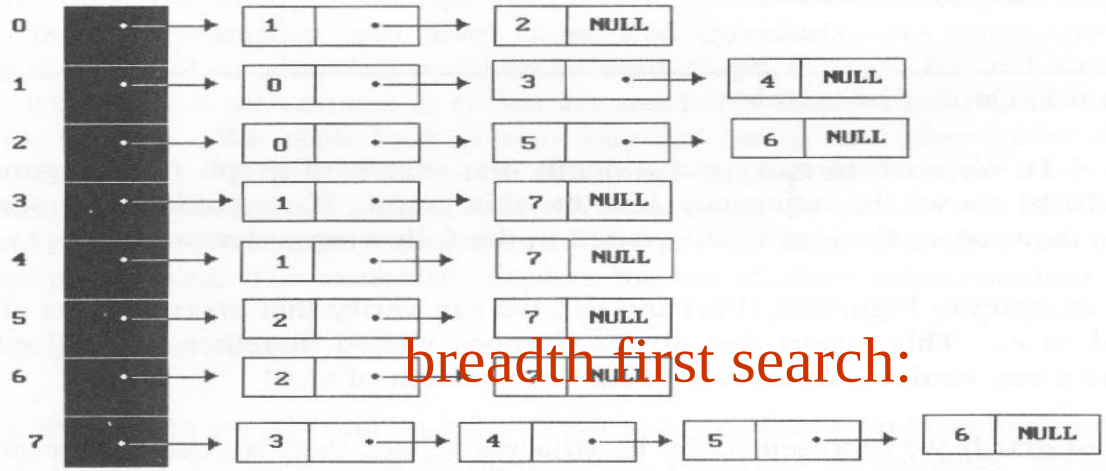
- Traversal
  Given G=(V,E) and vertex v, find all $w \in V$, such that w connects v.
  - Depth First Search (DFS)
  - Breadth First Search (BFS)
    level order tree traversal
- Connected Components
- Spanning Trees

*Figure 6.19: Graph *G* and its adjacency lists

depth first search:



(a)



(b)

breadth first search:

Depth First Search

```
short int visited[MAX_VERTICES];
```

```
void dfs(int v)
{
  node *w;
  visited[v]= true;
  cout << v;
  for (w=graph[v]; w; w=w->link)
    if (!visited[w->vertex])
      dfs(w->vertex);
}
```

Data structure
adjacency list: O(e)
adjacency matrix: $O(n^2)$

Breadth-First Search

```
void bfs(int v) {
        node *w;
        Queue queue;
        cout << v;
        visited[v]=true;
        queue.add( v);
        while(front) {
                v = queue.deleteq();
                for(w=graph[v]; w; w=w->link)
                        if(!visited[w->vertex]) {
                                cout <<  w->vertex;
                                queue.add( w->vertex);
                                visited[w->vertex] = true;
                        }
                }
        }
}
```

adjacency list: O(e)
adjacency matrix: $O(n^2)$

Connected component

- Is it a connected graph?
  - BFS(v) or DFS(v)
- Find out connected component

```
void connected(void){
int i;
for (i=0;i<n;i++){
        if(!visited[i]){
                dfs(i);
                cout << '\n';
        }
}
```

adjacency list: O(n+e)
adjacency matrix: $O(n^2)$

Breadth first search Algorithm

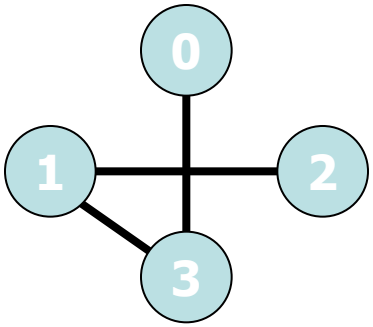
- Visiting first node and its neighbors of a graph.
- Set the status of all nodes to ready state, i.e. STATUS = 1.
- Put the starting node A into the queue and set its status to the waiting state.
- Repeat step-5 to 6 until queue is empty.
- Pick the front node of queue, process it and set its status to process state.
- Add all the neighbors of A into the queue that have status of ready state and set their status to the waiting state.
- [End of loop at step-4]
- exit

Depth first search

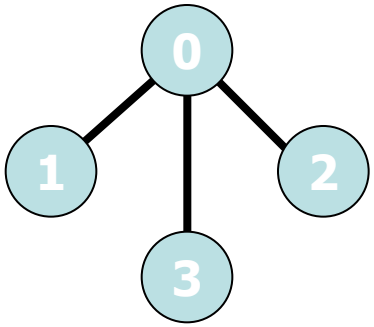
- Visting first node and its neighbor along the path P of a graph
- Set the status of all nodes to ready state, i.e. status = 1.
- Push the starting node A onto the stack and set its status to the waiting state.
- Repeat step 5 to 6 until stack is empty. Pop the top node of stack, process it and set its status to processed state.
- Push all the neighbors of A onto the stack that have status of ready state and set their status to the waiting state.
- [End of loop step 4]
- exit

Spanning Tree

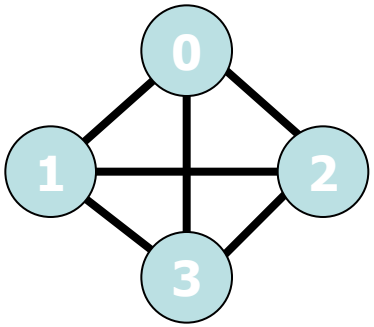
- A spanning tree is a minimal subgraph G', such that V(G')=V(G) and G' is connected
- Weight and MST



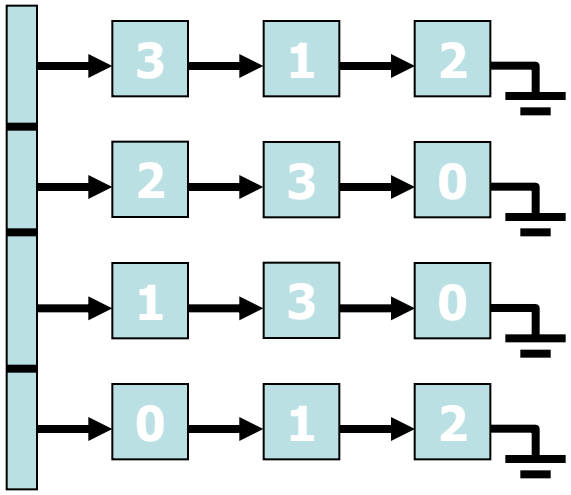
DFS(0)



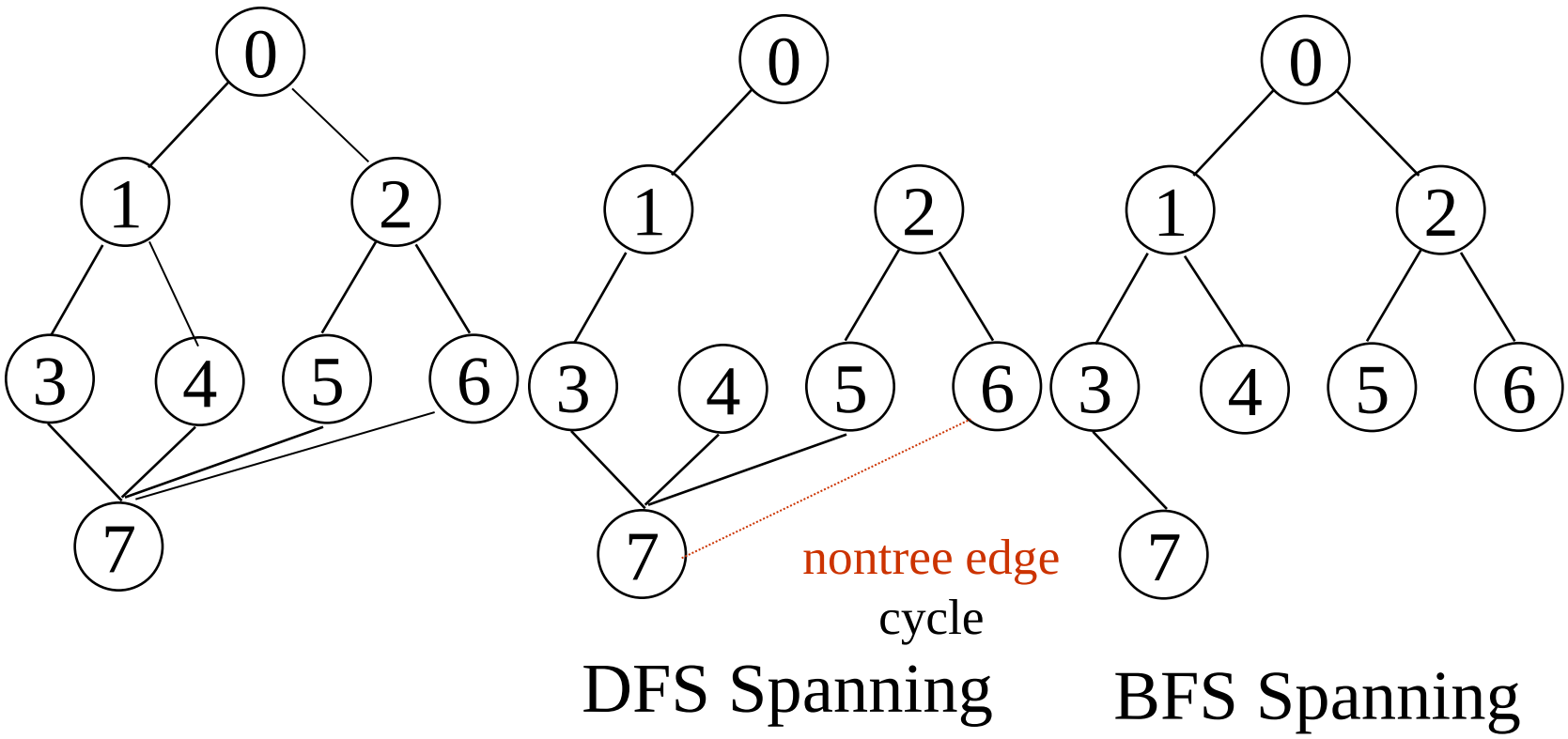
BFS(0)



G1



Spanning Trees

- Either dfs or bfs can be used to create a spanning tree
  - When dfs is used, the resulting spanning tree is known as a depth first spanning tree
  - When bfs is used, the resulting spanning tree is known as a breadth first spanning tree
- While adding a non-tree edge into any spanning tree, this will create a cycle

DFS VS BFS Spanning Tree